IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 22 January 2026, accepted 31 January 2026, date of publication 6 February 2026, date of current version 11 February 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3661078

RESEARCH ARTICLE

Implementation and Evaluation of a Time-Sensitive Networking Endpoint for Asynchronous Traffic Shaping

TAKAHIRO HIROFUCHI, AKRAM BEN AHMED, (Member, IEEE), AND TAKAAKI FUKAI

Intelligent Platform Research Institute (IPRI), National Institute of Advanced Industrial Science and Technology (AIST), Koto-ku, Tokyo 135-0064, Japan

Corresponding author: Takahiro Hirofuchi (t.hirofuchi@aist.go.jp)

This work was supported by the Research and Development Project of the Enhanced infrastructures for Post-5G Information and Communication Systems, commissioned by the New Energy and Industrial Technology Development Organization (NEDO) under Grant JPNP20017.

ABSTRACT Time-sensitive networking (TSN) is a set of standardized technologies that enable time-deterministic communication for latency critical applications. Asynchronous Traffic Shaping (ATS) is its advanced transmission algorithm defined in IEEE 802.1Q-2022. Although it has great advantages (e.g., the scalability of the number of latency-assured flows), there had been no implementation of ATS that actually works in the real world, until we presented the paper on the first functioning switch implementation and published its source code. To complete ATS support, however, an ATS-compliant endpoint implementation is still missing, which is the mechanism installed in a sending host to transmit frames in a manner complying with the ATS algorithm. In this paper, we propose the design and implementation of a TSN endpoint supporting ATS. As for the required functionality of an ATS endpoint, we employ the hardware/software co-design methodology. Only strictly timing-critical components are placed in hardware, thereby keeping the hardware mechanism as simple as possible. Flow-dependent components (e.g., the state machine to calculate the eligibility time of a frame) are placed in software, enabling scalability with respect to the number of ATS flows. It is not necessary to develop a dedicated hardware logic for an ATS endpoint. It is possible to use network interface cards with transmit timing control, which are available in the market. We implemented our prototype by using Intel i210 NIC and Linux, and confirmed through experiments that the transmit timing of ATS flows was accurately controlled according to the parameters of ATS flows. Even in a severe situation where 64 ATS flows competed at a sending host, the maximum observed contention delay was 86,574 ns, which is well below the theoretical upper bound of 787,889 ns derived from the ATS model.

INDEX TERMS Asynchronous traffic shaping, ATS, computer networks, ethernet, local area networks, network interfaces, real-time systems, scheduling algorithms, time-sensitive networking, timing jitter, TSN.

I. INTRODUCTION

Time-Sensitive Network (TSN) is a set of standardized technologies that enable time-deterministic communication over networks. It has garnered significant attention from both industry and academia [1], [2], [3]. TSN is based on the communication technology (i.e., Ethernet) that has been extensively used in computer networks. TSN facilitates higher-speed communications than conventional time-deterministic network technologies, such as Controlled Area Network [4]. TSN is beginning to be used in manufacturing control networks [5] and automotive in-vehicle networks [6]. Ongoing standardization efforts are also aimed at applying TSN to avionics [7] and 5G/6G's mobile fronthaul [8]. In addition, the combination of Multi-access Edge Com-

The associate editor coordinating the review of this manuscript and approving it for publication was Hosam El-Ocla.

puting (MEC) [9] and TSN is expected to make it possible to aggregate realtime sensing data over 5G/6G network and reliably perform cooperative distributed processing within a given short period of time. For example, sensor data from roadside units (RSU) installed at an intersection can be transferred to a MEC location (e.g., a regional datacenter in a 5G/6G radio access network). It generates a dynamic map around the intersection and transmits it to vehicles, enabling more advanced driver assistance (e.g., collision avoidance) than is possible with a single vehicle [10].[1]

TSN assures that end-to-end communication latency for a priority flow is under the upper bound value calculated theoretically. The end-to-end latency means the duration from when a sending host transmits a frame until the frame arrives at its receiving host via the series of network switches along the path of the flow. In the sending host and the network switches, the timing of frame transmission is controlled according to standardized algorithms, so that the period of time during which a frame of a priority flow is buffered in each switch is kept under a certain value. According to a mathematical model, the upper bound of the communication latency of a priority flow can be calculated based on the information on network topology and the flows in the network.

IEEE 802.1Q-2022 [11] defines several algorithms to control the transmit timing of frames. Asynchronous Traffic Shaping (ATS) is the most recently-defined algorithm that has advantages over the other algorithms. ATS does not require time synchronization among hosts and switches in a network, has no theoretical limitation on the number of priority flows whose worst-case latencies are assured, and supports burst transfer in each priority flow. However, there had been no implementation of ATS that actually works in the real world. In our prior work [12], we presented the hardware design of a TSN switch supporting ATS and published the source code of its FPGA implementation under an open source license [13]. As far as we know, this is the first and only available ATS implementation for a network switch. To complete ATS support, an ATS-compliant endpoint implementation is still missing, which is the mechanism installed in a sending host to transmit frames of priority flows in a manner complying with the ATS algorithm.

In this paper, we propose the design and implementation of a TSN endpoint supporting ATS. As for the required functionality of an ATS endpoint, we employ the hardware/-software co-design methodology. Only strictly timing-critical components are placed in hardware, thereby keeping the hardware mechanism as simple as possible. Flow-dependent components (e.g., the state machine to calculate the eligibility time (ET) of a frame) are placed in software, enabling scalability with respect to the number of ATS flows. It is not necessary to develop a dedicated hardware logic for an ATS endpoint. It is possible to use network interface cards with transmit timing control, which are available in the market. We implemented our prototype by using Intel i210 NIC [14] and Linux, and confirmed through experiments that the transmit timing of ATS flows was accurately controlled according to the parameters of ATS flows. The contention delays of ATS flows were correctly upper-bounded as calculated by its theoretical model. Even in a severe situation where 64 ATS flows competed at a sending host, the maximum observed contention delay was 86,574 ns, which remained well below the theoretical upper bound of 787,889 ns.

This paper focuses on the design and host-side validation of an ATS-compliant endpoint. While end-to-end evaluation over multiple ATS-capable switches is important for assessing system-wide scalability and comparative studies between ATS and other shaping mechanisms, it requires a holistic TSN testbed. We are currently developing such a testbed by integrating the proposed endpoint with our previously developed ATS-enabled switch. Multi-hop evaluation using this testbed is planned as future work.

The main contributions of this paper are summarized as follows:

- We present the first practical design and implementation of an ATS-compliant endpoint operating on actual hardware, addressing the absence of ATS endpoint support in existing NICs. To the best of our knowledge, no prior work has presented a practical ATS-compliant endpoint implementation operating on actual hardware, whereas our previous work provides the only available ATS-capable switch implementation.
- We address the unique endpoint design challenges posed by ATS, which regulates the transmission rate of individual flows while allowing multiple flows to share the same hardware queue. These challenges are fundamentally different from those of Scheduled Traffic (e.g., Time Aware Priority Shaper, TAPRIO in Linux), which controls transmission by time-gating hardware queues, Credit-Based Shaping, which enforces rate limits on a per-queue basis, and other coarse-grained shaping mechanisms such as Hierarchical Token Bucket (HTB) in Linux. While ETF (Earliest Tx-Time First) in Linux and the corresponding mechanism in DPDK (`send_on_timestamp`) provides transmission-timing control, it does not implement the ATS endpoint state machine or per-flow shaping behavior.
- We realize this novel endpoint architecture through a hardware/software co-design approach and demonstrate its feasibility by prototyping the endpoint on commodity hardware and quantitatively evaluating its timing accuracy and contention-delay behavior.

The rest of this paper is organized as follows. Section II gives the overview of TSN and ATS. Section III discusses design choices for ATS endpoints. In Section IV, we introduce the design of our ATS endpoint. Section V is dedicated

[1] This paper focuses on wired TSN mechanisms. The combination of wired ATS with wireless communication technologies, including its potential use in V2X and automated driving, are outside the scope of this work.

for performance evaluation. Section VI discusses related work. Finally, we end this paper with the conclusion in Section VII, followed by Appendix.

II. BACKGROUND

First, we provide an overview of how network devices (i.e., switches and endpoints) that support TSN work, introducing the notion of a Traffic Class (TC) and a transmission selection algorithm. Next, we explain ATS, which is one of the transmission selection algorithms defined in the IEEE standard.

A network device supporting TSN identifies a flow of frames, for example, by using the VLAN ID and the MAC addresses of the frames. The value of the Priority Code Point field (PCP) field in the VLAN header of a frame indicates its priority. Each transmit port of a network device has a maximum of 8 Traffic Classes (TC) to manage transmit frames according to their priorities. TC7 has the highest priority and TC0 has the lowest. A TC has a structure that holds pending frames before transmission. In some transmission algorithms, it is a structure other than a simple FIFO. In the IEEE standard, the structure of a TC is expressed as a queue system. The priority arbiter of a transmit port checks whether there is a frame ready for transmission in the TC with the highest priority, and if there is, the priority arbiter forwards the frame to the transmit port. If there is no frame that is ready for transmission in the TC, it attempts to find a frame ready for transmission in the second highest priority TC. This attempt is repeated from the highest to the lowest priority TC.

Each TC is assigned one of the transmission selection algorithms defined in the standard. Strict Priority (SP) is the simplest algorithm in which the head frame of the queue of the TC is always ready for transmission. This algorithm is typically set to the TCs with lower priorities. Best-effort flows whose worst-case latencies are not guaranteed are directed to these TCs. Credit-based Shaping is an algorithm to control the intervals of the frames sent from the TC. After a certain period of time has passed since the previous frame was sent from the TC, the head frame of the TC becomes ready for transmission. This control is done by counting up/down the state variable called Credit. It determines the time to wait until the next frame becomes ready for transmission, depending the size of the previous frame that was already transmitted and the transmit rate configured for the flow. Credit-based Shaping is set to higher-priority TCs, which accommodate priority flows with their worst-case latencies assured. Each TC assigned Credit-based Shaping can accommodate only one flow. Typically, TC7 and TC6 are used for Credit-based Shaping, which means that the number of latency-assured flows can be 2 or less. It is difficult to apply it for environments where multiple priority flows require latency assurance. It has a FIFO queue per flow and controls the transmit timing of the head frame of the queue. This mechanism is simple, but it requires implementing a separate queue for each priority flow. It is difficult to increase the number of supported latency-assured flows in a network device due to hardware resource limitations.

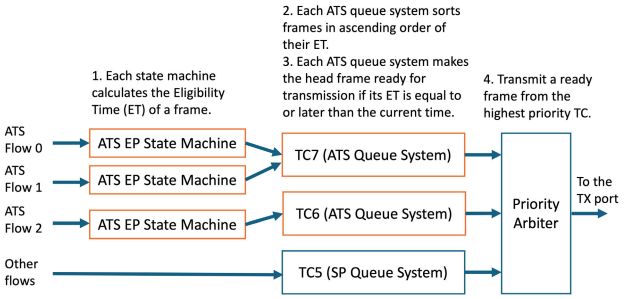

FIGURE 1. Overview of an ATS endpoint.

Asynchronous Traffic Shaping (ATS) is another transmission selection algorithm recently incorporated into the mainstream standard. ATS handles multiple flows in one TC, but substantially controls the frame interval of each flow, respectively. Thanks to the queue system called Interleaved Regulator, even though multiple flows share a single queue, the worst-case latency of each flow is guaranteed. Its theoretical model [15] proved that in the queue the impact of a flow to the latencies of the other flows is trivial, as long as these flows are correctly shaped at the previous hop of their network paths. A user can configure the maximum transmit rate called CommittedInformationRate (CIR) and the maximum burst size called CommittedBurstSize (CBS) for each flow. CBS allows transmitting the frames of a flow back-to-back within the configured transferred size.

Figure 1 shows an overview of an ATS endpoint. For each ATS flow transmitted from a sending host, the state machine of an ATS endpoint is implemented. The details of the state machine are defined in Clause 47.1.2 of IEEE 802.1Q-2022. See Appendix C for its pseudocode. The overview of the procedure is as follows:

- Step 1: When transmitting a frame of an ATS flow, the state machine calculates the Eligibility Time (ET) of the frame, and then forwards the frame to the ATS queue system of the TC that the flow belongs to.
- Step 2: The ATS queue system sorts pending frames in ascending order of their ET.
- Step 3: If the ET of the head frame is equal to or later than the current time, the queue system makes the frame ready for transmission.
- Step 4: The priority arbiter checks for the existence of a frame that is ready for transmission, starting from the highest-priority TC to lower TCs. If it exists, it transmits the frame.

The state machine of an ATS flow maintains its remaining burst size as its internal state.[2] The initial value of the

[2]The state machine stipulated in IEEE 802.1Q-2022 maintains the internal state that corresponds to the remaining burst size, in the time-domain variable called BucketEmptyTime. Although theoretically proved, it is not intuitive for readers to understand that the remaining burst size is expressed as BucketEmptyTime. Therefore, to ease of understanding, we use the remaining burst size in this paper.

remaining burst size is CommittedBurstSize. When an application calls the API to transmit a frame in an ATS flow, the frame arrives at the state machine. If the frame size is less than or equal to the remaining burst size, the ET of the frame is set to the arrival time of the frame, which means that the frame becomes ready for transmission right away. Otherwise, the ET is set to the future time at which the remaining burst size is recovered to the frame size. The recovering rate of the remaining burst size is based on CommittedInformationRate.

To ensure that the frames of an ATS flow meet their end-to-end latency requirements, the sending host needs to control the transmit timing of the frames according to the CommittedInformationRate and CommittedBurstSize of the ATS flow. All the switches along the path of the flow also need to control the transmit timing correspondingly.

There is a possibility that a frame is not transmitted exactly at the ET calculated by the state machine, due to contention with other frames in the ATS queue system and the priority arbiter. However, as long as all the flows in the same TC or higher-priority TCs are correctly controlled by ATS, the possible contention delay is always finite and its upper bound can be theoretically calculated.

III. DESIGN CHOICES

In order to assure the end-to-end latency of a flow by ATS, not only network switches but also sending hosts must be compatible with ATS. However, to the best of our knowledge, there is no endpoint implementation supporting ATS. No prior work in industry and academia reported that they succeeded in developing it. To address it, we discuss design choices for ATS endpoints and clarify the rationale for our co-design approach.

The first conceivable design choice is to implement all of Step 1 to 4 of the procedure in the software layer, leveraging the standard functionality of transmitting frames inherent to any network interface cards. However, this design choice cannot attain the requisite degree of accuracy in frame transmission required by ATS. TSN for Gigabit Ethernet (GbE) requires precise timing control of frame transmission with an accuracy of sub-microsecond order or less. A software program can attempt to control transmission timing, for example, by calling system calls such as `send()` and `nanosleep()`. However, it is difficult to control the time at which the frame is actually transmitted from the port of a NIC even with sub-millisecond precision. The time granularity of the task scheduling of an operating system is generally more coarse-grained. The timer interrupt frequency of Linux is typically configured to 250 Hz, corresponding to 4 ms of scheduling granularity. In addition, the DMA mechanism of NICs is optimized to transfer multiple frames at once. The operating system cannot directly control when the NIC retrieves the data of each transmit frame from the main memory and transmits it from the Ethernet port.

The second design choice is to implement all of Step 1 to 4 of the procedure in the hardware layer. Since we already implemented the FPGA-based TSN switch supporting ATS, we can reuse the part of its hardware design for an ATS endpoint. The design of the ATS queue system and the priority arbiter in the switch will be usable for the endpoint. The state machine for an ATS endpoint is partially different but rather simplified from that of the ATS switch. However, this design has the shortcoming that the number of the supported ATS flows is limited by the hardware resource size of an FPGA board. The contention between the ATS flows in the same sending host is allowed to exist only at the inside of an ATS queue system and the entrance of the priority arbiter. Thus, the ring buffer to transfer the frame of a flow from the CPU to the NIC, and the hardware queue to store the frame from the ring buffer need to be separated for each ATS flow. Based on our FPGA implementation of an ATS switch [12], even though a mid-class FPGA board costing around 3,000 USD is used, the ATS endpoint implementation can accommodate 2-3 ATS flows. Although there is room for optimization, the number of supported ATS flows barely scales.

The third design choice is to combine both the software and hardware mechanisms. The part of the procedure (i.e., Steps 3 and 4) that enables precise timing control of frame transmission is implemented in the hardware layer. The other part (i.e., Steps 1 and 2), where the number of supported ATS flows increases resource consumption, is implemented in the software layer. This enables frame-by-frame transmit required by ATS as well as scalability in terms of the number of supported ATS flows.

In this paper, we propose the system design of an ATS endpoint based on the third choice. We developed its prototype by using a commercial off-the-shelf NIC and evaluated its performance through experiments. Intel NICs such as i210 and i225 have the hardware functionality to control frame transmit time, which is called LaunchTime and supposedly designed to support Scheduled Traffic in IEEE 802.1Q-2022. A network controller chip of STMicroelectronics has the similar functionality called Time-based Scheduling (TBS). Similarly, several products in the series of NVIDIA's high-end datacenter NICs support time-triggered transmit scheduling of frames. Although it would be possible to develop our own FPGA-based NIC that implements Steps 3 and 4 in its hardware, doing so would require substantial development effort and long lead times. By contrast, implementing an ATS endpoint with a commercial off-the-shelf NIC will substantially reduce the lead time to put ATS into practical use in the real world.

With this in mind, we chose to use Intel i210 NIC for our prototype. Since its datasheet [14] is publicly available, it is suitable for prototyping and discussion. The proposed design is not specific to Intel i210 and can be applied to other NICs that provide equivalent transmit-time control functionality.

IV. DESIGN AND IMPLEMENTATION

Figure 2 shows the design of the ATS endpoint mechanism. For transmit timing control and priority arbitration, we conducted preliminary experiments to verify whether

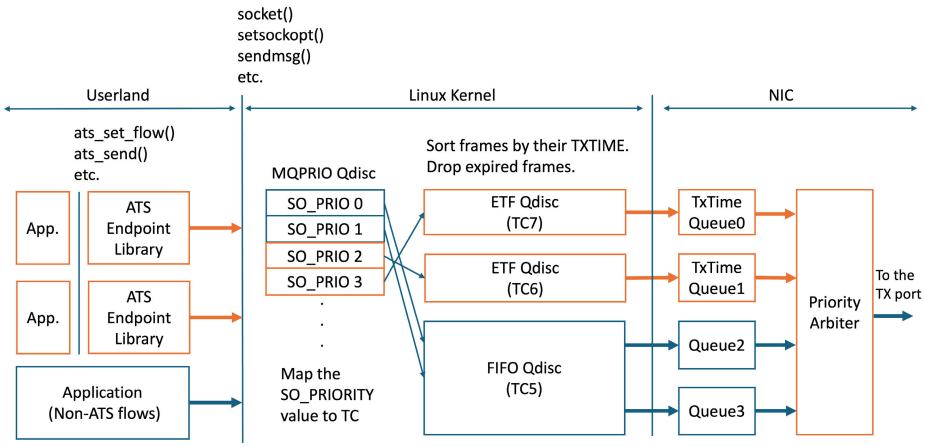

FIGURE 2. Design of the ATS endpoint.

the hardware functionality of Intel i210 complies with the behavior required by the IEEE standard. To correct the behavior of priority arbitration, we developed a patch for the Linux kernel driver. We also developed a mechanism to address a performance issue in transmit-timing control, which occurs under specific conditions. In the network stack of the Linux kernel, we made the queuing discipline that implements the array of TCs with different priorities. In the userland, we developed a programming library that implements the state machines for ATS flows.

A. NIC'S HARDWARE FUNCTIONALITY

We discuss whether the hardware functionality of Intel i210 is compatible with Steps 3 and 4 in the ATS endpoint defined in IEEE 802.1Q-2022. We conducted preliminary experiments for priority arbitration and transmit timing control, respectively.

1) PRIORITY ARBITRATION

Intel i210 has 4 hardware queues. If the NIC is configured to the mode to support Credit-based Shaping, called the Qav (IEEE 802.1Qav) mode, each queue has different priorities. Queue 0 has the highest priority and as the queue number increases the priority decreases. In the case that there is no frame ready for transmit in the higher priority queues, a frame ready for transmit in a queue can be transmitted. This behavior is the same as the priority arbitration among the TCs shown in Figure 1. In the Qav mode, the transmit timing control called LaunchTime in the datasheet can be enabled for Queue 0 and Queue 1.

In preliminary experiments, we observed that the priority arbitration of the NIC sometimes does not immediately retrieve a frame from Queue 1 even when there is no ready frame in Queue 0. This behavior is not compliant to the standard. The frames of ATS flows in the second or lower highest priority TCs will be unexpectedly delayed. The standard of the ATS endpoint demands that a frame ready for transmission in a TC must be retrieved immediately if no frame is being transmitted from any TCs and no other ready frame is pending in the higher-priority TCs. After thorough investigation, we figured out that we need to modify the device driver of Intel i210 to make its priority arbitration compatible with the standard. Although the expected behavior is not clearly documented in the datasheet, our experiments suggest that, as long as there are frames in Queue 0, the priority arbiter does not fetch frames from Queue 1, even if the frames in Queue 0 are not ready for transmission. We developed a kernel patch to fix this problem. It disables the SP_WAIT_SR flag in the control register for the Qav mode.

2) TRANSMISSION TIMING CONTROL

When sending a frame, the NIC driver prepares transmit descriptors in the ring buffer of the NIC. One descriptor contains the location and size of the frame in the host memory. When LaunchTime is used, another descriptor contains the transmit time of the frame. It is specified in 32-ns units in the LaunchTime field of the descriptor. The NIC initiates transmission at the specified time according to its internal clock counter called PTP (Precision Time Protocol) Hardware Clock (PHC). It maintains the current time of International Atomic Time (TAI[3]). The PHC ticks every 8 ns. See Appendix A for the detailed behavior of LaunchTime.

Through our preliminary experiments in Appendix A, we confirmed that Intel i210 NIC is basically capable of sending a frame at a given transmit time with an accuracy of 48 ns. Its accuracy can degrade, depending on the implementation of the PCIe bus of a sending host and also the frame size of a transmitted flow. In the design of the ATS endpoint, we developed a mechanism to adjust frame transmission times so as to keep safe frame intervals at which the accuracy of the transmit timing control does not deteriorate. In the evaluation section, when the NIC was attached to a PCIe Gen2 port, we adopted the value indicated by the minimum transmit interval with safety margin in

[3]The acronym is from Temps Atomique International.

```
/* Create a socket for an ATS flow.
   Close it. */
int ats_open_udp_socket(const char *
    ifname, int port, int
    enable_tx_report_errors);
int ats_close_udp_socket(int fd);

/* Initialize the state machine of the
   ATS flow. */
int ats_set_flow(int fd, __u64 cir,
    __u32 cbs, __u64 link_speed);

/* Send a frame in the ATS flow. */
int ats_sendmsg(int fd, void *data,
    size_t data_len, struct sockaddr_in *
    dest_addr);

/* Set the internal delay of the
   endpoint. */
int ats_set_processing_delay_max(__u64
    value);
```

FIGURE 3. The excerpt from the API of the ATS endpoint library.

Figure 18. It is simplified so that it increases linearly with respect to the frame size, while having a sufficient safe margin for any frame size. This linear model is calculated by adding 6,696 ns to the theoretical minimum frame interval of each frame size. See Section IV-C for how this model is incorporated into the state machine of the ATS endpoint.

B. QUEUING DISCIPLINE

Linux allows a userland program to specify the transmit time of an outgoing frame through the `SO_TXTIME` socket option and the `SCM_TXTIME` ancillary data. The ETF (Earliest TxTime First) queuing discipline (Qdisc) maintains these frames in ascending order of their transmit times. When the current time is a certain amount of time before the transmit time of a frame (i.e., 160 us before in the default setting of the prototype), ETF Qdisc dequeues the frame and hands it to the NIC driver, optionally leveraging the transmit timing control of NIC hardware.

Although ETF Qdisc is not designed for ATS, it provides the sorting functionality required for the ATS queue system. In our implementation, we construct TCs using the MQPRIO (Multi-queue Priority) Qdisc. TCs configured with ATS are mapped to ETF Qdiscs respectively and these ETF Qdiscs are mapped to separated hardware queues supporting timed transmission. TCs configured with SP are mapped to standard FIFO Qdiscs. In the experiments of this manuscript, we created 3 TCs: TC7 and TC6 configured with ATS, and TC5 with SP. The priorities of sockets (`SO_PRIORITY`) are mapped to these TCs, with priorities 3 and 2 mapped to TC7 and TC6, respectively. The other priorities are mapped to TC5, which does not assure worst latencies. See Appendix B for detail.

C. ATS ENDPOINT LIBRARY

We designed a programming library in the userland to allow an application to transmit the frames of ATS flows. It is in approximately 560 lines of code written in the C language. Figure 3 shows the excerpt from the API of the library. It provides an application with the abstracted interface to facilitate the creation of ATS flows, while it internally invokes system calls such as `socket()`, `setsockopt()` and `sendmsg()`. It calculates the ET of a frame according to the state machine of the ATS endpoint defined in the standard, and passes it to the kernel with its ET specified. It uses the file descriptor of a socket to distinguish the state machine of each ATS flow. See Appendix C for the pseudocode of the state machine.

An application first creates and initializes a socket for an ATS flow by `ats_open_udp_socket()`. Next, it specifies the CIR and CBS of the ATS flow by `ats_set_flow()`. It also specifies the link speed of the network interface, which is necessary for the calculation of the state machine. When the application sends a frame in the ATS flow, it calls `ats_sendmsg()`.

Although the application can call `ats_sendmsg()` anytime and repeatedly, the ET of each frame is calculated to meet the configuration parameter of the ATS flow. If the remaining burst size of the flow is under the size of a frame, the ET of the frame is delayed. If the socket send buffer in the kernel is full, the call to `ats_sendmsg()` is not completed until the queue has room for new frames.

As discussed in Section IV-A2, there is the possibility that the safe minimum transmit interval in the timing control mechanism of Intel i210 may change due to differences between hardware configurations and applications. The transmit timing control of Intel i210 can become unstable when the frame interval from the previous frame is shorter than the safe threshold depending on the size of a frame. In the library, if necessary, the state machine adjusts the ET of the frame to avoid the frame interval from being lower than the safe interval. Concretely, when calculating ET_i of the frame i, the state machine finds the safe frame interval I_i for the frame size, as modeled in the allowed shortest interval in Figure 18. If `ats_sendmsg()` for the next frame $i+1$ is invoked at the time ST_{i+1}, the arrival time AT_{i+1} of the next frame in the state machine is adjusted to $AT_{i+1} = max(ST_{i+1}, ET_i + I_i)$. If an application calls `ats_sendmsg()` consecutively and the burst size sufficiently remains, the behavior expected by the IEEE standard is to transmit frames back-to-back. In contrast, the behavior with this workaround is to transmit frames with the safe interval inserted. At the inside of the endpoint host, a frame of an ATS flow may experience additional delay due to the workaround. However, its upper bound can be modeled if the timing of when transmit data is generated by an application is known. In addition, the contention delay of the flow caused in the network switches along the path to its destination is not adversely affected. Since the ATS endpoints with this workaround moderate burst transfers of ATS flows in the network, the contention delay of each ATS flow due to competing flows is rather mitigated.

TABLE 1. Summary of evaluation objectives and metrics.

Section	Evaluation scenario	Purpose	Metrics	Figures
V.A	CIR shaping test	Verify that the ATS endpoint correctly shapes traffic for different CIR settings	Measured frame interval, derived throughput, derived frame jitter	Figs. 4–7
V.B	CBS shaping test	Verify that the ATS endpoint correctly enforces different CBS settings	Measured frame interval	Figs. 8–10
V.C	Inter-class priority test	Verify that the ATS endpoint correctly enforces priority among different traffic classes	Measured frame interval, derived throughput	Figs. 11, 12
V.D	Contention delay test	Verify that the ATS endpoint correctly bounds the contention delay of ATS flows	Measured latency (difference between eligibility time (ET) and actual transmit time)	Figs. 13–16

As a minor note, as discussed earlier, LaunchTime of Intel i210 must be a multiple of 32 ns. If an ET is not a multiple of 32 ns, our library rounds it up. Although the transmit time of the frame is slightly delayed, it is within the deviation of the transmission timing control (i.e., 48 ns).

V. EVALUATION

We evaluated the proposed ATS endpoint mechanism through experiments. Table 1 summarizes the objectives, evaluation scenarios, and metrics of each experimental section. We developed test application programs using the ATS endpoint library, which sets up ATS flows and repeatedly transmits their frames with various configurations. The used machines were equipped with an Intel CPU i7-13700. To obtain reproducible results, the Dynamic Voltage and Frequency Scaling (DVFS) of the CPU core was disabled to fix its frequency at its maximum speed. Its temporal sleep mechanism (i.e., Intel C-state) was also disabled. The Active State Power Management (ASPM) of PCI Express was disabled by default and remained disabled. The experimental environment was based on Ubuntu 22.04 LTS running Linux kernel 5.15.168. In some experiments, `iperf3` version 3.7 was used as a traffic generation tool.

A. CIR

First, we confirmed that the ATS endpoint mechanism correctly controls the frame intervals of an ATS flow according to any CIR assigned to the flow. In the same manner as Section IV-A2, the Intel i210 NIC on the sending host was directly connected to the Intel i210 NIC on the receiving host. The Intel i210 NIC was capable of recording the arrival times of received frames with a measurement granularity of 8 ns. We developed a test application program for experiments, which generates the frames of an ATS flow with its specified parameters. In this experiment, it repeatedly invoked `ats_sendmsg()` without any sleep for 10 seconds.[4] The frame size in the flow was fixed to 1518 bytes. The CIR of the ATS flow was set to a different value each time. The CBS was set to the frame size, which means that the burst transfer was disabled.

[4]Even for the lowest CIR setting of 50 Mbps, more than 40,000 frames were measured, resulting in 95% confidence intervals of approximately ± 0.01–0.06 ns for reported mean values.

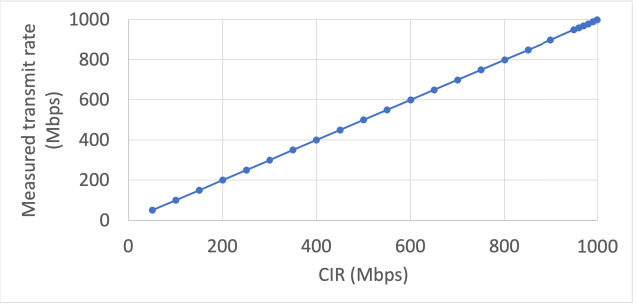

FIGURE 4. Measured transmit rate of the ATS flow as a function of the configured CIR, obtained with the Intel i210 NIC attached to a PCIe Gen4 slot.

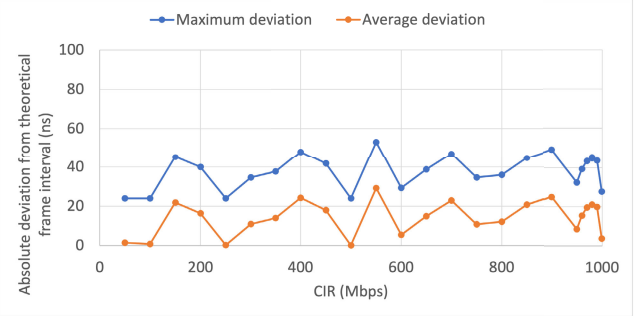

FIGURE 5. Maximum and average deviation of the observed frame intervals from the theoretical interval for each CIR, measured with the Intel i210 NIC attached to a PCIe Gen4 slot.

The Intel i210 NIC of the sending host was attached to the PCIe Gen4 slot of the host. As shown in Figure 4, the actual transmit rate of the ATS flow, which was calculated by the observed arrival times at the receiving host, matched to the configured CIR. In Figures 5, the maximum deviation from the theoretical interval for each CIR was under 60 ns. The average deviation was under 30 ns.

Next, the NIC was attached to a PCIe Gen2 slot of the host. As in Figures 6 and 7, with the CIR of 600 Mbps or less, the actual transmit rate matched to it, and the deviation from the theoretical interval was the same as that of the Gen4 slot. However, even though the CIR was set to 650 Mbps or more, the actual transmit rate was capped at 647 Mbps, not following the CIR. The safe interval adjustment for the Gen2 slot assures transmit frame intervals to be no less than 19,000 ns for the frame size of 1518 bytes, which corresponds to the transmit rate of 647 Mbps at most.

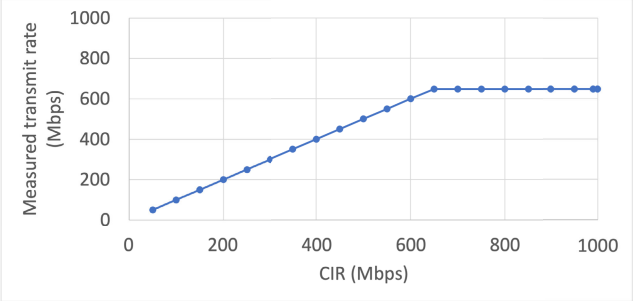

FIGURE 6. Measured transmit rate of the ATS flow as a function of the configured CIR, obtained with the Intel i210 NIC attached to a PCIe Gen2 slot.

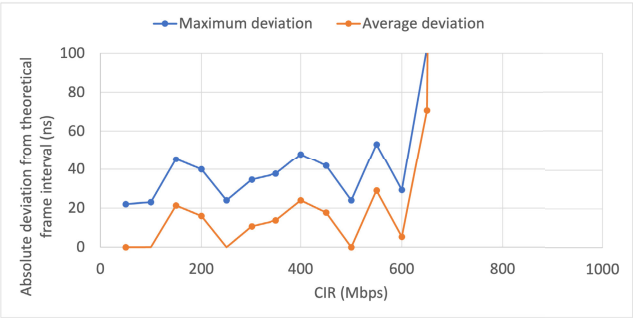

FIGURE 7. Maximum and average deviation of the observed frame intervals from the theoretical interval for each CIR, measured with the Intel i210 NIC attached to a PCIe Gen2 slot.

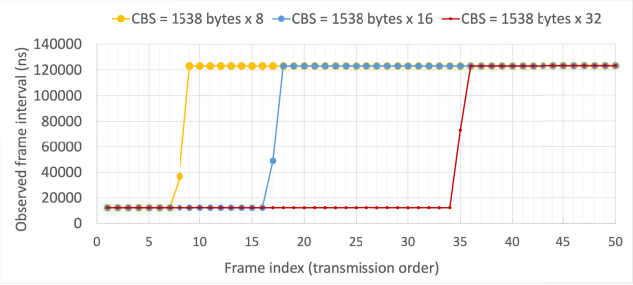

FIGURE 8. Observed frame intervals for different CBS values, measured with the Intel i210 NIC attached to a PCIe Gen4 slot. The CBS was configured as a multiple of the physical layer frame size (1538 bytes). The lines overlap at Y ≈ 12,000 and 120,000 ns.

In summary, we confirmed that the ATS endpoint mechanism precisely controlled transmit intervals to be aligned with specified CIRs. The observed deviations from the theoretical intervals were sufficiently small, barely impacting the end-to-end latency of an ATS flow that is orders of magnitude larger (e.g., a few milliseconds even in a small network). Even though the Gen2 slot of the used machine caused instability in the transmit timing control of Intel i210, the safe interval adjustment in the ATS endpoint mechanism alleviated this problem. It was usable as long as the CIR was set to 600 Mbps or less.

B. CBS

Second, we evaluated the capability to control the burst transfer of an ATS flow according to its CBS. The test

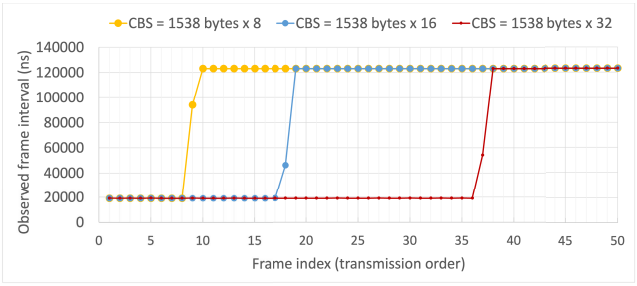

FIGURE 9. Observed frame intervals for different CBS values, measured with the Intel i210 NIC attached to a PCIe Gen2 slot. The CBS was configured as a multiple of the physical layer frame size (1538 bytes). The lines overlap at Y ≈ 19,000 and 120,000 ns.

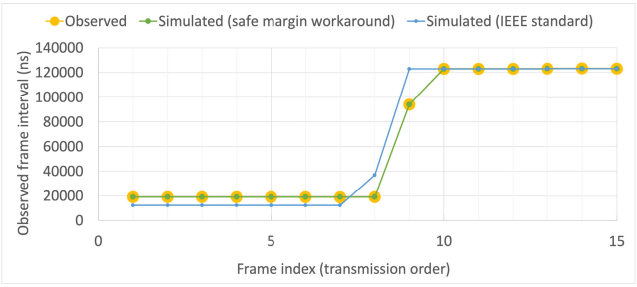

FIGURE 10. Observed and simulated frame intervals for CBS = 1538 × 8 bytes, measured with the Intel i210 NIC attached to a PCIe Gen2 slot. The simulated results include the standard ATS behavior and the safe margin workaround.

application program set up an ATS flow at CIR 100 Mbps, using a different CBS each time. In the flow, the frame size was fixed to 1518 bytes, which corresponds to 1538 bytes at the physical layer including Preamble, Start Frame Delimiter and Interframe Gap. The CBS was set to a multiple of this physical layer frame size (i.e., 1538 × N bytes, where N is a positive integer). The program repeatedly invoked `ats_sendmsg()` 100 times for the ATS flow without any sleep.

Figures 8 and 9 show observed frame intervals with different CBS values. The Intel i210 NIC was attached to the Gen4 slot or the Gen2 slot, respectively. For the latter case, the workaround to keep safe frame intervals, noted in Section IV-C, was enabled.

In Figure 8, when the sufficient burst size remained for the frame size, the sending host transmitted the frames at intervals of approximately 12,300 ns with any tested CBS. After the burst size was exhausted, it transmitted the frames approximately every 123,000 ns, which was the duration of the burst size recovery at CIR 100 Mbps for the frame size.

As shown in Figure 9, with the safe margin workaround enabled for the Gen2 slot, the observed short frame intervals was around 19,000 ns, which was the safe shortest interval for the frame size in the linear model discussed in Section IV-A2. Figure 10 details the result with the CBS of 1538 × 8 bytes. We also obtained through simulations the theoretical frame intervals with the workaround for Intel i210 and the ones expected by the standard. The ATS endpoint mechanism accurately controlled frame transmission timing as expected.

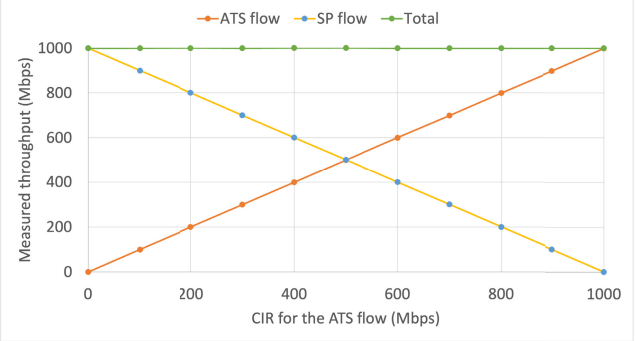

FIGURE 11. Measured throughput of an ATS flow and a Strict Priority (SP) flow, as a function of the CIR configured for the ATS flow. The results were obtained with the Intel i210 NIC attached to a PCIe Gen4 slot.

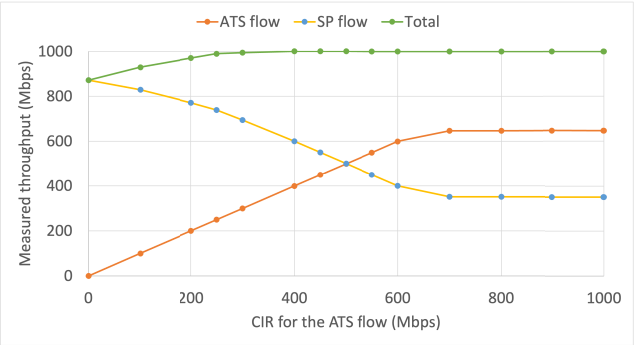

FIGURE 12. Measured throughput of an ATS flow and a Strict Priority (SP) flow, as a function of the CIR configured for the ATS flow. The results were obtained with the Intel i210 NIC attached to a PCIe Gen2 slot.

The observed intervals matched to the simulation results calculated theoretically incorporating the workaround. The difference between them at each frame interval was only approximately 3 ns on average. As also shown in the graph, the observed frame intervals with the sufficient burst size were greater than the 12,304 ns expected by the IEEE standard for the physical frame size of 1538 bytes. This result is consistent with the discussion in Section IV-C. In this study, we validated the CBS behavior by examining the initial 50 frames, which was sufficient to confirm the correctness of the burst shaping behavior given the deterministic nature of the CBS configuration and the transmit timing accuracy verified in other experiments (i.e., up to approximately 60 ns deviation).

C. PRIORITY ARBITRATION

We evaluated the priority arbitration of the ATS endpoint mechanism. Although multiple flows compete at the transmission port, the endpoint is expected to prioritize frame transmissions for ATS flows over other flows. The strict arbitration is necessary to bound the contention delays that occur in the ATS flows.

The test application program was used to generate an ATS flow through the highest-priority TC (i.e., TC7), with different CIRs assigned. Simultaneously, a frame generator program `iperf3` was used to continuously transmit a best-effort flow of frames, called a SP flow, through the lowest-priority TC assigned SP in the endpoint (i.e., TC5). The frame size of both the flows was 1538 bytes at the physical layer. Each measurement lasted for 10 seconds, during which even at the lowest CIR of 100 Mbps, an ATS flow transmitted approximately 800,000 frames. Without any other competing flows, the SP flow achieved 1 Gbps and 872 Mbps at the physical layer with the Gen4 slot and the Gen2 slot, respectively, in the Qav mode. The throughput through the Gen2 slot in the Qav mode was degraded from that of the non-Qav mode (i.e., 1 Gbps) due to the hardware limitation discussed in Section IV-A2.

Figures 11 and 12 show the actual transmission throughput of the flows, which was measured at the receiving host. With the Gen4 slot, the ATS flow achieved the same throughput as the CIR at any tested CIRs. As the CIR of the ATS flow increased from 0 Mbps, the throughput of the SP flow decreased correspondingly. This suggests that the arbiter correctly prioritized the ATS flow. In the ATS flow, the maximum observed deviation from the scheduled frame interval was 12,312 ns, which closely aligned with the theoretically expected maximum of 12,304 ns, differing by only 8 ns. If a frame in the SP flow was being transmitted at the time when a frame in the ATS flow was scheduled to be sent, the frame in the ATS flow was delayed until the transmission of the frame of the SP flow was completed. With the Gen2 slot, when the CIR was 600 Mbps or less, the ATS flow achieved the same throughput as the CIR. For the CIR of 700 Mbps or more, the ATS flow achieved 647 Mbps regardless of CIR. This result was consistent with Section V-A.

We also conducted the experiments to observe the priority arbitration between ATS flows of different priorities. Instead of the SP flow, we used the second highest ATS flow from TC6, and obtained the same graph except that SP was replaced with the second highest ATS.

In summary, we confirmed that the priority arbitration of the ATS endpoint correctly worked for different priority flows. The high-priority ATS flow achieved the same transmission rate as its CIR, and the low-priority SP or ATS flow obtained the remaining bandwidth. Frames in the high-priority flow can be delayed by low-priority flows, but its maximum delay is bounded by the transmission time of the maximum frame size in the low-priority flows.

D. CONTENTION DELAY

In the above experiments, we confirmed that the ATS endpoint mechanism was capable of controlling frame intervals according to the priority, CIR and CBS of an ATS flow. For this rectified flow, the network switches along its path assure the worst-case contention delays that occur at their TCs and priority arbiters. In order to assure the end-to-end latencies over network, we also need to confirm that the ATS endpoint mechanism can bound the contention delay of the ATS flow inside its sending host. It is the period of time from the ET of a frame until the NIC actually transmits the frame from its port.

TABLE 2. Latency (ns) of two competing ATS flows, measured for each frame and summarized by the average with 95% confidence interval, maximum, minimum, and standard deviation. ATS0 and ATS1 belong to traffic classes TC7 and TC6, respectively, with TC7 having higher priority. The results were obtained with the Intel i210 NIC attached to a PCIe Gen4 slot.

	ATS0 (TC7, CIR 100 Mbps)	ATS1 (TC6, CIR 101 Mbps)
Ave	1,050 ±13	1,053 ±13
Max	12,725	12,756
Min	423	423
Std	2,157	2,165

TABLE 3. Latency (ns) of two competing ATS flows—coexisting with a SP flow of 700 Mbps as background traffic—measured for each frame and summarized by the average with 95% confidence interval, maximum, minimum, and standard deviation. ATS0 and ATS1 belong to traffic classes TC7 and TC6, respectively, with TC7 having higher priority. The results were obtained with the Intel i210 NIC attached to a PCIe Gen4 slot.

	ATS0 (TC7, CIR 100 Mbps)	ATS1 (TC6, CIR 101 Mbps)
Ave	7,597 ±27	8,242 ±34
Max	14,285	26,584
Min	412	412
Std	4,420	5,558

To record the actual transmission time of each frame with high precision, we connected an Intel i210 NIC of a host to the other i210 NIC of the same host via a 30-cm Ethernet cable. The sender NIC transmitted frames and the receiver NIC recorded the arrival time of each frame by using its receive hardware timestamp. We regarded the arrival time of a frame recorded at the receiver NIC as the transmission time of the frame (i.e., the time when the sender NIC sends the first byte of the frame from its port). Since the latency of the cable is diminutive and constant, which is 1-2 ns with the typical NVP (Nominal Velocity of Propagation) value (e.g., 70%) of Ethernet cables, we ignored it for ease of discussion. It should be noted that although Intel i210 NIC supports transmit hardware timestamp, it can record the transmission time of a frame as long as only the frame exists in its hardware queue. It is usable for a single sporadic flow with up to 100 frames per second [12], but is not applicable for multiple fast flows in this experiment. This limitation is supposedly because it is designed for sporadic PTP synchronization messages.

The ATS endpoint library uses the system clock synchronized with International Atomic Time (i.e., `CLOCK_TAI`) to calculate the ET of each frame. We synchronized the system clock with the PTP hardware clock inside the sender NIC by a utility program (`phc2sys`) in the PTP support for Linux (LinuxPTP 4.4). It maintained the offset between the clocks within 50 ns in a stable state, according to its output. We also synchronized the PTP hardware clock of the sender NIC with that of the receiver NIC by connecting their external signal pins via a jumper wire. The NIC configured as the source side generated a pulse from its signal pin at each clock tick occurring exactly at 000 ms 000 us 000 ns. The NIC at the sink side recorded its PTP hardware clock upon the arrival of the pulse. If the recorded clock did not align exactly with 000 ms 000 us 000 ns, the hardware clock was adjusted accordingly. The program used for synchronization (`ts2phc`) reported an adjustment of 10 ns or less every second. To verify the accuracy of synchronization, we also measured the offset between the source and sink clocks by using the PTP daemon program `ptp4l` with the measurement-only option (i.e., no clock adjustment by PTP). We confirmed that the offset consistently remained 30 ns or less once a stable state was reached. Thus, we consider that it is possible to discuss latency measured on the order of 100 ns.

Detailed validation with two competing ATS flows:

First, we launched 2 test application programs to generate 2 competing ATS flows. The CIRs of the first and second ATS flows (ATS0 and ATS1) were set to 100 Mbps and 101 Mbps, respectively. The former flow was assigned the highest priority, sent from TC7. The latter flow was assigned the second highest priority, sent from TC6. We intentionally chose the different CIRs for the flows to create the contentions of frame transmissions. In each flow, 100,000 frames were generated. Their frame size was 1538 bytes at the physical layer, and their CBS was also set to 1538 bytes. See Appendix D for the calculation of the upper bound contention delay of an ATS flow.

We present the results with the PCIe Gen4 slot, since we obtained similar results with the Gen2 slot. As shown in Table 2, the shortest latency of frames was 423 ns, which was observed when no contention happened between the transmission timings of the ATS flows. It is considered as the base latency between the sender and receiver NICs. The maximum latencies were 12,725 ns and 12,756 ns in ATS0 and ATS1, respectively. These values were consistent with the base latency plus the theoretical worst contention delay (i.e., 423 ns plus 12,304 ns).

Second, we added an SP flow sent from TC5. The frame generator program transmitted frames at the rate corresponding 700 Mbps at the physical layer. In theory, the maximum contention delay of the ATS flow of the highest priority is 12,304 ns. That of the second highest priority is 24,608 ns. Just after the first byte of a frame in the SP flow starts being sent, a frame in ATS0 and a frame in ATS1 can become ready for transmission. In this case, the frame of ATS1 needs to wait for the transmissions of the other 2 frames, resulting the double contention delay.

As shown in Table 3, the averages of the observed latencies in ATS0 and ATS1 were higher than those of the previous experiment, because the added SP flow incurred more frequent contentions. The observed maximum latencies (i.e., 14,285 ns and 26,584 ns) were fairly close to the expected values (i.e., 12,714 ns and 25,016 ns, calculated with the base latency of 412 ns). However, the observed maximum latencies were approximately 1,500 ns higher than expected. Although not observed in the experiment of Section V-C, we consider that in particular situations, the NIC's behavior can be slightly deviated from the arbitration model demanded by the standard. Since the datasheet of the NIC does not describe the detail of the arbitration behavior, it is difficult to identify the root cause that possibly exists in the NIC.

We also conducted the same experiments as those reported in Tables 2 and 3, with the difference that both the ATS flows

were transmitted from TC7, and obtained proper results. Without the SP flow, the maximum latencies were 12,754 ns and 12,752 ns in ATS0 and ATS1, respectively. With the SP flow, they were 26,414 ns and 26,528 ns. Because the ATS flows are in the same priority, a frame in one of the ATS flows can be delayed by that of the other ATS flow. Thus, these maximum latencies were similar to that of the second highest priority ATS flow in Table 3.

Scalability evaluation with increasing numbers of competing ATS flows:

Finally, we increased the number of competing ATS flows. In order to create various degrees of contentions, we set a different CIR to each ATS flow. Up to 7 competing ATS flows, the CIR of the n-th ATS flow was set to $100 + n$ Mbps (e.g., 100 Mbps for ATS0 and 101 Mbps for ATS1). The bandwidth of the SP flow was 100 Mbps at the physical layer. Figure 13 shows the results of the observed maximum latency of ATS0 with up to 7 competing ATS flows. Without the competing SP flow, the observed maximum latency was correctly bound by the theoretical maximum calculated according to the mathematical model detailed in Appendix D, even when many competing ATS flows coexisted. In the cases with 4 competing ATS flows and more, the observed maximum latency did not reach the theoretical upper bound, because of the rare chance of such severe contentions. With the SP flow, for the small numbers of the competing ATS flows, the observed maximum latency slightly exceeded the theoretical upper bound, which was the same deviations as those reported in Table 3. Except that, the latency of the ATS flow was successfully bounded by the theoretical bound.

Figure 14 shows the results with up to 63 competing ATS flows. Note that to accommodate up to 64 ATS flows and the 100-Mbps SP flow in the 1-Gbps link, the CIR of the n-th ATS flow was set to $10 + 0.1\ n$ Mbps. With any numbers of competing ATS flows, the observed maximum latency was successfully under the theoretical bound. As the number of ATS flows increased, the observed maximum latency also increased; however, its growth was more gradual than that of the theoretical upper bound. Although more severe contention delays can occur when the eligibility times of a larger number of flows become closely aligned, such severe contention events become increasingly unlikely. In the case where an ATS flow competed with 63 other ATS flows and the coexisting SP flow, the maximum observed latency was 180,956 ns, which corresponds to contention with at least 15 frames. The ATS endpoint mechanism guaranteed that this maximum latency did not exceed the theoretical upper bound (787,889 ns).

To detail the distribution of observed latencies, Figure 15 shows the latency histogram and the corresponding cumulative distribution for the case where ATS0 contends with 7 other ATS flows and the coexisting SP flow. Table 4 summarizes the key statistical metrics of the observed latency distribution, including selected percentiles. Figure 16 and Table 5 present the corresponding results for the more demanding case where ATS0 contends with 63 other ATS flows and the coexisting SP flow.

In both cases, the 99th-percentile frames are observed to have latencies of approximately half of the maximum observed latency. Similarly, the 99.9th-percentile frames remain within roughly 70% of the maximum latency, indicating that extreme contention delays occur only rarely even under high concurrency.

In the experimental setup, the test application program runs for each ATS flow. Consequently, in the former case, a total of 9 active processes are executed on the sending host (8 ATS test programs and one `iperf` program for the SP flow) at least, whereas in the latter case, 65 active processes run concurrently. Due to the effects of the task scheduling of the host operating system, the time interval between the computed eligibility time and the actual transmission of a frame may fluctuate. We consider that the pronounced peaks around 16,000 ns and 25,000 ns observed in the histogram of the former case are attributable to this scheduling effect.

TSN aims at providing worst-case latency guarantees, and the absence of frame loss for priority flows is a fundamental requirement. Within the scope of our experiments, no frame loss was observed for ATS flows even in intentionally stressed scenarios designed to evaluate scalability, namely with 64 concurrent ATS flows and coexisting background traffic. Such a level of concurrency will be unrealistically high for a single endpoint in practical deployments, but was deliberately chosen to examine worst-case behavior.

In this scenario, five frame drops were observed in the background traffic generated by `iperf`. This behavior is acceptable, as the background traffic belongs to a lower-priority flow that does not require worst-case latency guarantees. Our analysis suggests that these drops did not occur in the Linux queuing discipline layer, but were more likely attributable to the hardware layer.

The CPU utilization of each test application program generating an ATS flow was approximately 0.3%. Even at the highest level of concurrency evaluated, the overall system CPU utilization remained around 5.1%, indicating that the system operated with sufficient processing headroom. Nevertheless, to reliably deploy the proposed ATS endpoint in real-world environments, it is essential to provision adequate system resources so that sufficient CPU time is allocated to applications generating ATS flows. In addition, care must be taken to ensure that the Linux kernel's network processing is not inadvertently delayed by other interrupt handling or system activities. These considerations are not unique to the proposed ATS endpoint, but are common to host-based implementations of other TSN mechanisms, including Credit-based Shaping and Scheduled Traffic.

VI. RELATED WORK

Table 6 provides a concise summary of representative related works and positions our proposed ATS-compliant endpoint with respect to prior studies.

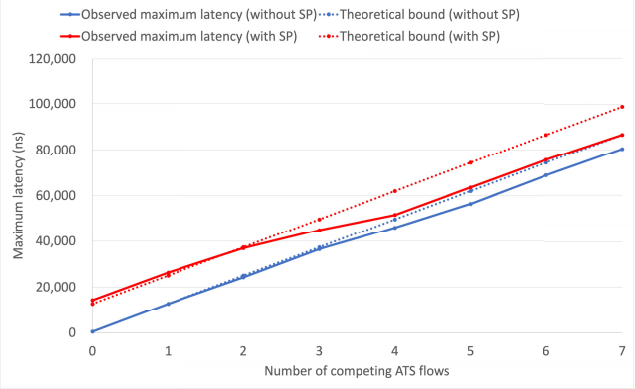

FIGURE 13. Observed maximum latency and its theoretical bound of an ATS flow as a function of the number of competing ATS flows, evaluated with and without a coexisting SP flow. The results include experiments with up to–7 competing ATS flows–and were obtained using the Intel i210 NIC attached to a PCIe Gen4 slot.

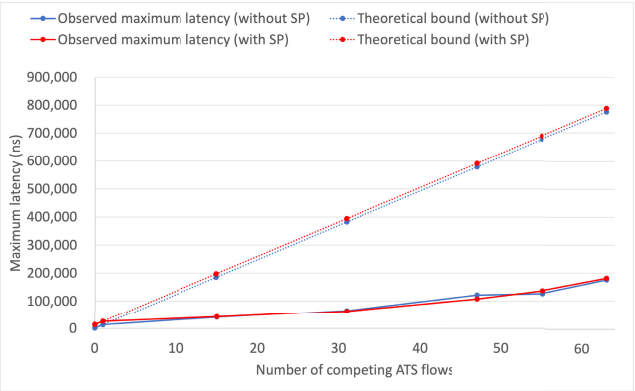

FIGURE 14. Observed maximum latency and its theoretical bound of an ATS flow as a function of the number of competing ATS flows, evaluated with and without a coexisting SP flow. The results include experiments with up to–63 competing ATS flows–and were obtained using the Intel i210 NIC attached to a PCIe Gen4 slot.

TABLE 4. Latency statistics (ns) of the ATS flow contending with 7 other ATS flows and a coexisting SP flow, corresponding to the histogram in Figure 15.

Scenario	Statistic	Latency (ns)
7 ATS flows + SP flow	Minimum	433
	Mean ±95% CI	14173 ±65
	Median (50th)	12743
	95th percentile	34022
	99th percentile	43405
	99.9th percentile	52964
	Maximum	86574

According to a survey paper [1], reflecting the growing expectations for IoT and 5G/6G, the number of publications in the TSN domain has been increasing since the late 2010s. Research is shifting from the theoretical aspects of control algorithms to studies aimed at practical application. However, the majority of existing research is confined to theoretical formulations and simulation-based analysis. At the time of the survey conducted in 2020, only 6% of studies (i.e., 5 out of 93 studies) on transmission algorithms had been experimentally validated on actual hardware, and no studies

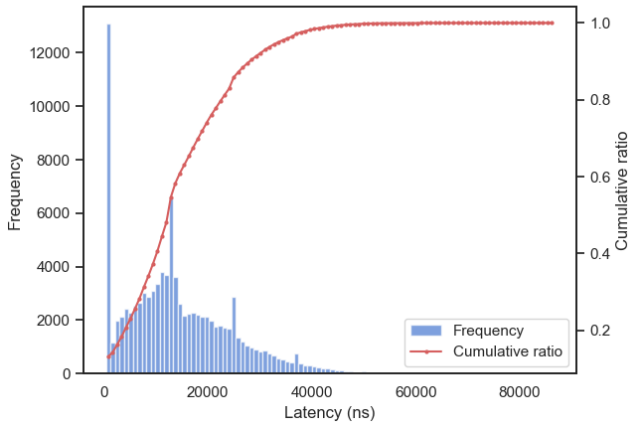

FIGURE 15. Latency histogram and its cumulative distribution of the ATS flow under contention with 7 other ATS flows and a coexisting SP flow, measured with the Intel i210 NIC attached to the PCIe Gen4 slot. The histogram is composed of 100 bins. The width of each bin is 861.41 ns.

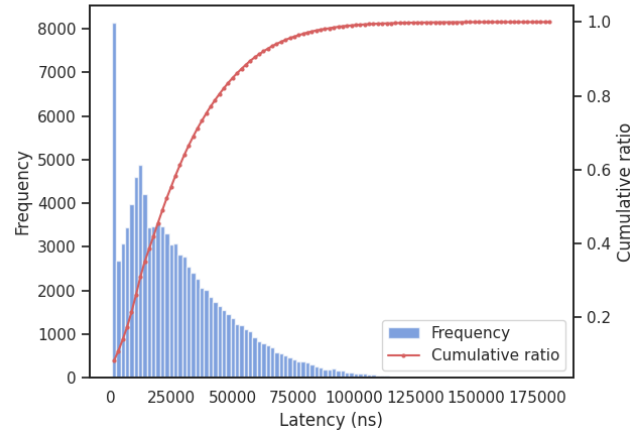

FIGURE 16. Latency histogram and its cumulative distribution of the ATS flow under contention with 63 other ATS flows and a coexisting SP flow, measured with the Intel i210 NIC attached to the PCIe Gen4 slot. The histogram is composed of 100 bins. The width of each bin is 1805.38 ns.

TABLE 5. Latency statistics (ns) of the ATS flow contending with 63 other ATS flows and a coexisting SP flow, corresponding to the histogram in Figure 16. The mean.

Scenario	Statistic	Latency (ns)
63 ATS flows + SP flow	Minimum	418
	Mean ±95% CI	27876 ±140
	Median (50th)	22622
	95th percentile	72092
	99th percentile	98191
	99.9th percentile	131838
	Maximum	180956

on Credit-based Shaping and ATS on actual hardware were reported. We consider that this trend seems to persist even today.

As for Credit-based Shaping, commercialization has been ongoing now. Its implementations can be found in commercially available network switches and endpoints.

TABLE 6. Summary of representative related works and comparison with this work.

Work	Target	Method	Keypoints
TSN research trends [1]	TSN domain	Survey	Reports that most studies are theory/simulation-based; limited hardware validation.
CBS switch (our prior work) [16]	Switch (CBS)	Implementation	First work discussing Credit-based Shaping (CBS) switch hardware design with fully working open-source code.
UBS [15] and equivalence proof [17]	Algorithm (ATS)	Theory	Provides theoretical basis; ATS in IEEE 802.1Q-2022 uses BucketEmptyTime as a time-domain state to facilitate hardware implementation.
Performance studies [18]–[24] on ATS	Network (ATS, etc)	Simulation (in-house or OMNeT++ [25])	Validates performance in simulation; does not capture practical implementation constraints on real systems.
ATS switch (**our prior work**) [12]	Switch (ATS)	Implementation	First functioning ATS switch implementation on actual hardware with published source code.
Partial ATS switch [26]	Switch (ATS)	Implementation	Implements part of ATS scheduler; does not design and validate a complete ATS-capable switch.
TSN testbed [27]	Network (Scheduled Traffic)	Implementation	Reports ATS support in some NICs/boards; based on a conceptual endpoint using HTB [28], which we argue is not ATS-compliant (coarse-grained shaping).
Linux TAPRIO	Endpoint (Scheduled Traffic)	Implementation	Scheduled Traffic (TAS) requires network-wide time synchronization and scheduling; complementary to ATS; if combined, reduces contention delay.
TSN testbed [29]	Network (Scheduled Traffic and CBS)	Implementation	Report Intel i210's LaunchTime behavior and configuration trade-offs; provides baseline knowledge for timing control on i210.
Hardware-assisted transmit scheduling on Linux [30]	Endpoint (Scheduled Traffic)	Implementation	Implements Scheduled Traffic (TAS) using Intel i210; not Linux's TAPRIO; reports transmit-time variation of 81 ns, comparable to our 48 ns.
Hardware-assisted transmit scheduling on DPDK [31]	Endpoint (generic transmit-timing scheduling)	Implementation	Implements transmit scheduling using Intel i210 and the packet processing framework; lower latency; possible to port our ATS endpoint to it but sacrifices socket API compatibility.
Software-based transmit scheduling [32]	Endpoint (generic transmit-timing scheduling)	Implementation	Implements transmit scheduling with normal NICs; incur bandwidth/CPU overhead due to the placeholder technique; possible to implement our ATS endpoint on it but not suitable for fast or bursty ATS flows.
ATS endpoint (**This work**)	Endpoint (ATS)	Implementation	First ATS endpoint design and implementation on actual hardware; per-flow ATS shaping while sharing hardware queues, with quantitative validation against theoretical delay bounds; prototyped with the hardware assist of Intel i210.

For example, Intel i210 NIC implements it in its hardware logic, and the Linux network stack implements the queuing discipline of Credit-based Shaping (CBS Qdisc), which supports the offload option to use the hardware logic in the NIC. However, there had been no academic paper that discussed how Credit-based Shaping should be implemented in the hardware logic of a network switch. Our prior work is the first paper [16] on the design of a TSN switch supporting Credit-based Shaping, along with its fully working source code under an open source license.

ATS is a more advanced algorithm, which was incorporated into the mainline of the IEEE standard in 2022. The ATS algorithm defined in the standard is based on the Token Bucket Emulation (TBE) algorithm of the Urgency-based Scheduler (UBS) [15] published in 2016. It was theoretically proved that both the algorithms were exactly equivalent [17]. While TBE uses the remaining burst size of each ATS flow, ATS uses a state variable called BucketEmptyTime, which is a time-domain representation of the remaining burst size, to facilitate hardware implementation. Existing studies [18], [19], [20], [21], [22], [23], [24] discussing the performance of transmission algorithms including ATS employed their in-house simulation program or a simulation framework such as OMNeT++ [25]. Although these simulation-based studies contributed to validating the performance of ATS, there is the inevitable gap between a simulation and the real world. They do not fully capture implementation constraints and practical considerations. There had been no implementation running on actual hardware until we presented the paper on the first functioning switch implementation [12] and published its source code. Before that, only the study on the implementation of ATS was the one reported by [26], which presented a partial FPGA implementation of the ATS scheduling algorithm without designing the entire switch. There was no prior work presenting a comprehensive idea on how an ATS-capable switch should be designed.

For the network interface of a host machine, to the best of our knowledge, there is no endpoint implementation supporting ATS, to our knowledge, either in academia or in commercial products. A survey paper [33] presented the list of commercial products supporting various TSN features. They reported that Intel i210/225 NIC, Kontron PCIe-4000 NIC and InnoRoute RealTime HAT (i.e., add-on board for Raspberry Pi) support ATS in combination with the Linux

network support. However, we consider that this information does not correctly reflect the current circumstances. They referred to a short position paper [28] as the basis for its assertion that the ATS implementation exists. Although the position paper claims that Hierarchical Token Bucket (HTB) Qdisc in Linux is usable to implement the ATS algorithm, we consider that it is not possible to implement the standard-compliant behavior of ATS. Unlike ATS, HTB does not strictly schedule individual frame transmissions. Although HTB has the `burst` and `rate` configuration parameters, it performs traffic shaping in a coarse-grained manner that is far from the exact frame-by-frame basis control demanded by the standard.

The IEEE standard allows that the transmission selection algorithms including ATS are used with the Scheduled Traffic mechanism that controls the periodical transmission window time of each traffic class. It is also known as Time-aware Shaping (TAS) and IEEE 802.1Qbv (i.e., the name of the amendment used until being merged into the main 802.1Q standard). Scheduled Traffic requires precise time synchronization among all the endpoints and switches in a network. It also needs to solve complex mathematical optimization. Although those aspects are disadvantages of Scheduled Traffic, if it is used in combination with ATS, it can mitigate the contention delays that ATS flows experience along their end-to-end paths. The Linux network stack supports a simplified version of the Scheduled Traffic with TAPRIO (Time Aware Priority Shaper) Qdisc. Our ATS endpoint implementation is complementary to TAPRIO and it is possible to combine both the mechanisms.

Several studies discussed the performance of the transmit timing control of Intel i210 NIC and also its application to the design of an endpoint [29], [30], [31]. As background, upon the transmission of a frame, the NIC retrieves the frame data from the main memory to its on-chip memory via the PCIe bus, and then the MAC/PHY module sends the frame data onto the wire sequentially. To precisely control the transmit timing, the frame data needs to arrive at the on-chip memory before its LaunchTime. ETF Qdisc has the configuration parameter called `delta`. ETF Qdisc schedules a callback routine to deliver the frame over to the underlying NIC driver at its TxTime minus `delta`. The study [29], presenting a framework for reproducible TSN experiments, reported that the latencies of frames linearly increased as `delta` was increased. They chose 175 us for it since the jitter of latencies was the lowest. For our ATS endpoint implementation, it is necessary to set a sufficiently large value of `delta` so that a scheduling delay of the execution of the callback routine does not affect the operation. However, when multiple ATS flows share a TC, a large `delta` value can cause frame drops because ETF Qdisc does not accept a new frame if its TxTime is less than the TxTime of the last frame already sent to the underlying NIC driver. In our setting, an appropriate value for `delta` was 160 us, which balanced the trade-off without involving the problems.

Reference [30] proposed their own implementation of Scheduled Traffic, which is different from Linux's TAPRIO but using LaunchTime of the NIC. They reported that when LaunchTime was disabled, the transmission interval varied by approximately 22 us, whereas enabling LaunchTime reduced the variation to approximately 81 ns. As mentioned in Section IV-A2, we observed a variation of 48 ns in our preliminary experiment, which closely aligns with the reported 81 ns variation.

A position paper [31] discussed the design of a TSN endpoint not relying on the Linux network stack. They used a userland network programming framework called DPDK (Data Plane Development Kit) and prototyped a mechanism to schedule frame transmissions by the NIC. Although the setup of the experiment to measure end-to-end latency is not detailed, they claim that the use of DPDK contributed to 2 times lower latency than the Linux network stack. We consider that it is possible to implement our ATS endpoint also by using DPDK. Although the compatibility with the majority of network applications using the socket API is not preserved, the programming model of DPDK to exclusively use particular CPU cores for each network task can mitigate latency fluctuations that occur at a sending host. Their subsequent paper [32] proposed a software-based technique to schedule frame transmissions on commodity NICs without dedicated hardware support. The authors claim compatibility with Linux APIs for transmission time control, suggesting potential applicability to our ATS endpoint mechanism. However, because the method relies on fixed-length placeholder descriptors to manage outgoing traffic, it incurs additional bandwidth and CPU overhead, which may limit its applicability to ATS flows with high CIR values or burst transfers.

VII. CONCLUSION

We developed a TSN endpoint supporting ATS, which is based on the hardware/software co-design methodology. Only strictly timing-critical components are placed in hardware, thereby keeping the hardware mechanism as simple as possible. Flow-dependent components (e.g., the state machine to calculate the eligibility time of a frame) are placed in software, enabling scalability with respect to the number of ATS flows. It is not necessary to develop a dedicated hardware logic for an ATS endpoint. It is possible to use network interface cards with transmit timing control, which are available in the market. We implemented our prototype by using Intel i210 NIC and Linux, and confirmed through experiments that the transmit timing of ATS flows was accurately controlled according to the parameters of ATS flows. Even in a severe situation where 64 ATS flows competed at a sending host, the maximum observed contention delay was 86,574 ns, which remained well below the theoretical upper bound of 787,889 ns.

While the proposed ATS endpoint design itself does not impose inherent limitations, the prototype implementation using Intel i210 NIC revealed several practical constraints

when applied as an ATS endpoint. As discussed in Section IV-A2 and Appendix A, the accuracy of transmit timing control depends on the performance of the PCIe bus to which the NIC is attached, and the timing precision can degrade under certain PCIe configurations. In addition, as shown in Section V-D, minor deviations from the behavior required by the IEEE specification were observed in priority arbitration under contention. These limitations are attributed to the characteristics of the commercial off-the-shelf NIC used in the prototype, rather than to the proposed endpoint architecture itself. As future work, we plan to develop a custom FPGA-based NIC optimized for ATS endpoints in order to fully satisfy the standard requirements and to eliminate these hardware-induced limitations.

We also plan to conduct end-to-end experiments using a holistic TSN testbed that integrates the ATS endpoint presented in this paper with the ATS-capable switch from our prior work. This will allow us to evaluate scalability, hop-by-hop latency accumulation, and feasibility at the network-wide level. This environment will also enable comparative studies between ATS and other TSN shaping mechanisms such as Scheduled Traffic (TAS) and CBS under realistic configurations. We have also developed an FPGA-based frame generator and capture for TSN research, called EFCC (Ethernet Frame Crafter and Capture) [34]. It can generate a pre-defined, complex sequence of frames mixed with various ATS and non-ATS flows and measure their end-to-end latencies in a 8-ns precision. This allows us to conduct thorough investigations on the performance of ATS in the wild in a reproducible manner. Our upcoming publication will summarize these efforts and report the latest results.

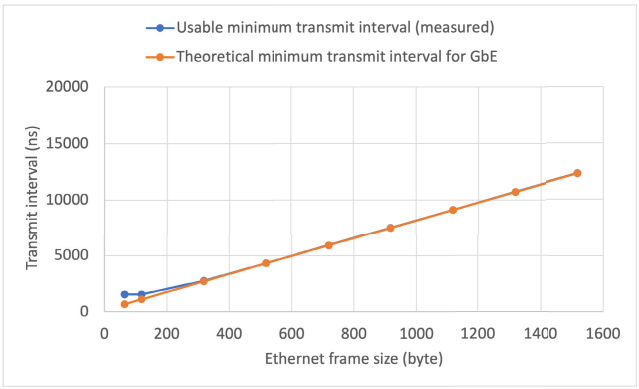

FIGURE 17. Frame transmit intervals (ns) at which accurate transmission timing control was achieved, with the Intel i210 NIC attached to a PCIe Gen4 slot. The usable minimum transmit interval indicates the smallest transmission interval at which the deviation between the maximum and minimum received frame intervals remains below 100 ns.

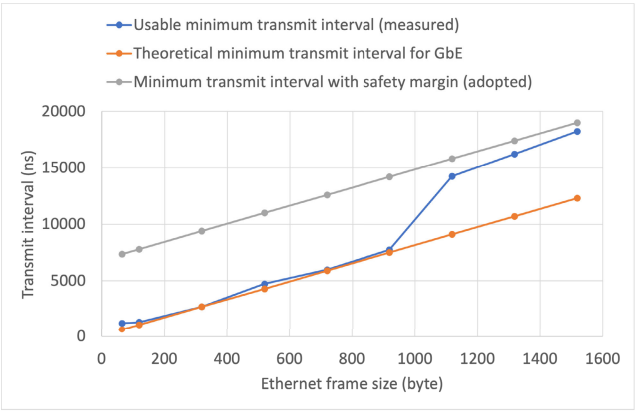

FIGURE 18. Frame transmit intervals (ns) at which accurate transmission timing control was achieved, with the Intel i210 NIC attached to a PCIe Gen2 slot. The usable minimum transmit interval indicates the smallest transmission interval at which the deviation between the maximum and minimum received frame intervals remains below 100 ns. The minimum transmit interval with safety margin is used for the ATS endpoint.

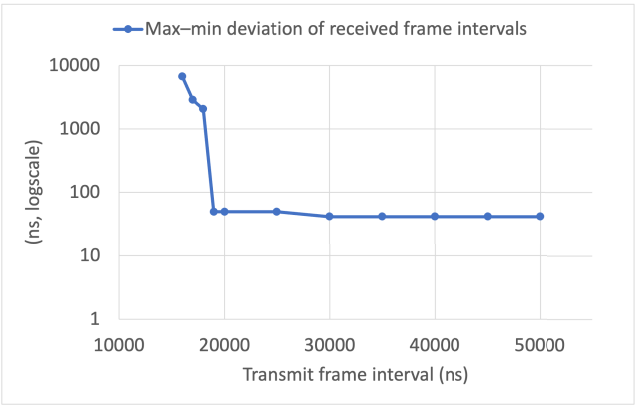

FIGURE 19. Max-min deviation of received frame intervals as a function of transmit frame interval for a frame size of 1518 bytes, measured with the Intel i210 NIC attached to a PCIe Gen2 slot.

APPENDIX A
DETAILS OF LaunchTime IN INTEL i210 NIC

A. OVERVIEW

Intel i210 NIC uses two types of descriptors in its transmit ring buffer. When transmitting a frame, the operating system adds a new entry called Transmit Data Descriptor (TDD) in the transmit ring buffer of the hardware queue that the frame is transmitted from. The TDD contains, for example, the physical address in the main memory where the data of the frame exists, and its data size. If controlling the transmit time of the frame by using LaunchTime, the operating system also adds in the transmit ring buffer a new entry called Transmit Context Descriptor (TCD) before the entry of the TDD.[5]

If the transmit time of a frame specified by the operating system is the current time or earlier, the NIC transmits the frame immediately. If the transmit time is later than the current time, it holds the frame until then. This comparison is made using the digits below the seconds of each time. The range within −0.5 seconds of the current time is treated as the past, and the range under +0.5 seconds of the current time is treated as the future.

B. ACCURACY

As a preliminary experiment, we evaluated the transmit timing control of Intel i210 NIC. We found that it is basically

[5]TCD is also used to control checksum and segmentation offloads. They are not used in this paper.

capable of transmitting a frame with an accuracy of 48 ns, although the accuracy is likely to degrade under certain conditions. For the setup of the experiment, we directly connected the Intel i210 NIC of a sending host to the Intel i210 NIC of a receiving host using an Ethernet cable. Intel i210 is equipped with the receive hardware timestamp that can record frame receive times in nanoseconds. Its internal clock is updated every 8 ns. See Section V for the details of the used machine.

The sending host attempted to transmit frames via the Intel i210 NIC at regular intervals of a specified value for 5 seconds. For a frame size of 1518 bytes, this corresponds to approximately 400,000 transmitted frames, which is sufficient to evaluate the maximum variation in frame intervals. The receiving host recorded their arrival times. We tested different frame sizes from the shortest one to the standard maximum (i.e., 64 bytes to 1518 bytes).

First, we attached the NIC to a PCIe Gen4 × 16 slot of the main board of the sending host. The PCIe physical layer of Intel i210 NIC is based on PCI Express 1.1 (Gen1) and the bus width is 1 lane (x1). Although the PCIe slot is capable of Gen4 × 16, the NIC linked up at Gen1 × 1. See Figure 17. The usable minimum transmit interval refers to the smallest transmission interval at which the deviation between the maximum and minimum received frame intervals remains below 100 ns. This threshold indicates the range in which the NIC maintains stable timing accuracy.

If the frame size was 518 bytes or larger, the NIC accurately transmitted frames at intervals equal to the smallest multiple of 32 ns that exceeds the theoretical minimum interval determined by the frame size. For example, with the frame size of 1518 bytes, the theoretical minimum interval is 12,304 ns at GbE. Because Intel i210 controls the transmit time of a frame in 32-ns units as mentioned above, 12,320 ns was the smallest controllable transmit interval. With this transmit interval, the average received frame interval was 12,320.07 ns. The fluctuation in the received frame intervals was only 48 ns. For all the tested frame sizes between 518 bytes and 1518 bytes, the fluctuation was 48 ns, which is sufficiently small to implement the ATS endpoint for GbE for most use cases.

For the frame sizes of 318 bytes or smaller, however, the NIC failed to control the transmit timings at the smallest controllable transmit interval. For example, with the frame size of 64 bytes, 672 ns is the theoretical minimum and also a multiple of 32 ns. However, in this case, most transmitted frames were dropped at the sending host. The transmit times of the dropped frames were already expired at the entry of ETF Qdisc. We found that 1,536 ns was the minimum stable interval at which no frame drop was observed. The fluctuation in the received frame intervals was also 48 ns in this case. We consider that, on the machine used in the experiments, the frame processing in ETF Qdisc was not able to keep up with the maximum frame rate of the short frame size.

Second, we attached the NIC to a PCIe Gen2 × 1 slot of the sending host. See Figure 18. For the frame sizes of 918 bytes or smaller, it was possible to accurately transmit frames at intervals close to the theoretical minimum. However, for the larger frame sizes, the fluctuation in the received frame intervals drastically degraded at such short transmit intervals. Thus, we adopted the value indicated by the minimum transmit interval with safety margin for our ATS endpoint. See Section IV-A2.

Figure 19 shows the max-min deviation in received frame intervals in the case that the frame size of 1518 bytes. When the transmit frame interval was less than approximately 18,000 ns, the accuracy degraded markedly. It was not possible to accurately control transmit timings at intervals close to the theoretical minimum (i.e., 12,320 ns). For example, with the transmit frame interval of 16,000 ns, 5.3% of the received intervals involved more than 100-ns deviations. It should be noted that if the Qav mode was disabled, the NIC achieved the theoretical maximum throughput of sending frames without the transmit timing control. We observed that enabling the Qav mode changes how the NIC fetches entries from the ring buffer. It will fetch only a limited number of entries in advance and its performance becomes sensitive to the latency of the PCIe bus. On the machine used in our experiments, the latency of Gen2 slots from the CPU will be slightly larger than that of PCIe Gen4 slot.

APPENDIX B
DETAIL OF QUEUING DISCIPLINES

A. TIMED TRANSMISSION

In Linux, for the network socket with the `SO_TXTIME` option enabled, an application can specify the transmit time (TxTime) of a frame by attaching `SCM_TXTIME` ancillary data to a sendmsg() call. The transmit time is stored in the kernel's socket buffer object (i.e., `skb->tstamp`).

Linux implements the queuing discipline called ETF (Earliest TxTime First) Qdisc in its network stack, which handles the frames with TxTime specified. ETF Qdisc maintains queued frames in ascending order of their transmit times. When the current time is a certain amount of time before the transmit time of a frame, ETF Qdisc dequeues the frame for transmission and passes it to the NIC driver. If the current time passes the transmit time of the frame, ETF Qdisc discards it. This frame drop typically occurs when the application generates frames faster than the line rate of the NIC. If the NIC hardware supports the transmit timing control, we can enable the hardware offload option for ETF Qdisc. The transmit time is programmed into the transmit ring buffer of the NIC.

B. PRIORITY MAPPING

MQPRIO (Multi-queue Priority) Qdisc controls the mapping among socket priorities, Qdiscs and hardware queues. We configured MQPRIO to construct multiple queues corresponding TCs of TSN.

- A TC assigned ATS is mapped to an ETF Qdisc attached to a hardware queue with timed-transmission capability.

- A TC assigned SP is mapped to a standard FIFO Qdisc attached to one or more normal hardware queues. Frames are distributed among multiple queues. In the used Linux kernel (5.15.168), the standard FIFO Qdisc is Fair Queuing Controlled Delay (FQ-CoDel).

The application can specify the priority of outgoing frames via `SO_PRIORITY`. In the experiments, the following mapping was configured:

- Socket priority 3: TC7 (ATS)
- Socket priority 2: TC6 (ATS)
- Other priorities and the default: TC5 (SP)

APPENDIX C
ELIGIBILITY-TIME COMPUTATION FOR ATS ENDPOINT

For completeness and clarity, this appendix shows the pseudocode of the state machine in our ATS endpoint implementation. It calculates the eligibility time (ET) of an ATS frame as defined in IEEE 802.1Q-2022, Clause 47.1.2. The state variable BucketEmptyTime is denoted as *BET* and initialized to zero. The ATS parameters CommittedBurstSize and CommittedInformationRate are denoted as *CBS* and *CIR*, respectively. Abbreviated symbols are introduced only for the state variable, the ATS parameters and input variables, while intermediate variables are written using descriptive names for clarity.

Algorithm 1 Eligibility Time *ET* Computation for Frame i at an ATS Endpoint

Input: frame length L_i, arrival time AT_i
State: BucketEmptyTime (*BET*, initialized to 0)
Parameters: CommittedBurstSize (*CBS*), CommittedInformationRate (*CIR*)
$lengthRecoveryDuration \leftarrow L_i/CIR$
$emptyToFullDuration \leftarrow CBS/CIR$
$schedulerEligibilityTime \leftarrow BET + lengthRecoveryDuration$
$bucketFullTime \leftarrow BET + emptyToFullDuration$
$ET_i \leftarrow \max(AT_i,\ schedulerEligibilityTime)$
if $ET_i < bucketFullTime$ **then**
    $BET \leftarrow schedulerEligibilityTime$
else
    $BET \leftarrow schedulerEligibilityTime + ET_i - bucketFullTime$
end if

APPENDIX D
LATENCY BOUND CALCULATION FOR ATS ENDPOINT

The upper bound of contention delay at an ATS endpoint is calculated based on the same theoretical model as that used for ATS switches [12], [34]. For a given traffic class, the upper bound of contention delay $d_{CO}(h)$ for flow h is obtained by computing the value $d(f)$ defined in Equation 2 for each flow f belonging to the same traffic class $F_S(h)$ as flow h, and then taking the maximum among these values.

$$d_{CO}(h) = \max_{f \in F_s(h)} d(f) \tag{1}$$

$$d(f) = \frac{\sum_{g \in F_H(f) \cup F_S(f)} b_{max}(g) - l_{min}(f) + l_{LP,max}(f)}{R - \sum_{g \in F_H(f)} r_{max}(g)} \tag{2}$$

where:

- $\sum_{g \in F_H(f) \cup F_S(f)} b_{max}(g)$: The sum of the maximum burst sizes of all the flows in higher traffic classes $F_H(f)$ and those in the same traffic class $F_S(f)$
- $l_{min}(f)$: The minimum frame size of the flow f
- $l_{LP,max}(f)$: The maximum frame size of any lower traffic classes
- R: The link speed of the network interface
- $\sum_{g \in F_H(f)} r_{max}(g)$: The sum of the maximum transmit rates of all the flows in higher traffic classes $F_H(f)$

For example, in the last experiment of the contention delay evaluation presented in Section V-D, all ATS flows belonged to the highest traffic class. The physical-layer frame size was fixed to 1538 bytes for all flows, including the SP flow belonging to a lower traffic class. The committed burst size (CBS) of each ATS flow was set to 1538 bytes, and the link speed was 1 Gbps.

Let n denote the number of ATS flows. Under these conditions, when the SP flow is present, the upper bound of the contention delay of an ATS flow h is given as follows:

$$d_{CO}(h) = \frac{1538 \times n - 1538 + 1538 \ \text{bytes}}{1 \ \text{Gbps}}. \tag{3}$$

REFERENCES

[1] Y. Seol, D. Hyeon, J. Min, M. Kim, and J. Paek, "Timely survey of time-sensitive networking: Past and future directions," *IEEE Access*, vol. 9, pp. 142506–142527, 2021.
[2] Y. Xu and J. Huang, "A survey on time-sensitive networking standards and applications for intelligent driving," *Processes*, vol. 11, no. 7, p. 2211, 2023.
[3] T. Fedullo, A. Morato, F. Tramarin, L. Rovati, and S. Vitturi, "A comprehensive review on time sensitive networks with a special focus on its applicability to industrial smart and distributed measurement systems," *Sensors*, vol. 22, no. 4, pp. 1–30, Feb. 2022.
[4] *Road Vehicles—Controller Area Network (CAN). Part 1: Data Link Layer and Physical Coding Sublayer*, Standard ISO 11898-1:2024, International Organization for Standardization, 2024.
[5] *IEC/IEEE Draft International Standard Time-Sensitive Networking Profile for Industrial Automation*, Institute of Electrical and Electronics Engineers, Washington, DC, USA, 2023.
[6] *IEEE Standard for Local and Metropolitan Area Networks–Time-Sensitive Networking Profile for Automotive In-Vehicle Ethernet Communications*, Standard 802.1DG-2025, Institute of Electrical and Electronics Engineers, 2025.
[7] *Draft Standard for Local and Metropolitan Area Networks: Time-Sensitive Networking for Aerospace Onboard Ethernet Communications*, Standard IEEE 802.1DP-2025, Institute of Electrical and Electronics Engineers, 2025.
[8] *IEEE Standard for Local and Metropolitan Area Networks—Time-Sensitive Networking for Fronthaul*, Standard 802.1CMde-2020, Institute of Electrical and Electronics Engineers, 2018.
[9] *Multi-Access Edge Computing (MEC); Framework and Reference Architecture*, European Telecommunications Standards Institute, Sophia Antipolis, France, Jun. 2025.

[10] *Multi-Access Edge Computing (MEC); Use Cases and Requirements*, European Telecommunications Standards Institute, Sophia Antipolis, France, Jun. 2025.
[11] *IEEE Standard for Local and Metropolitan Area Networks–Bridges and Bridged Networks*, Standard 802.1Q-2018, Institute of Electrical and Electronics Engineers, 2022.
[12] A. Ben Ahmed, T. Hirofuchi, and T. Fukai, "Hardware design and evaluation of an FPGA-based network switch supporting asynchronous traffic shaping for time sensitive networking," *IEEE Access*, vol. 12, pp. 123149–123165, 2024.
[13] (2024). *The AIST-TSN Project Repository*. [Online]. Available: https://github.com/CCIRT/aist-tsn/
[14] *Intel Ethernet Controller I210 Datasheet, Revision Number: 3.7*, Intel Cooperation, Santa Clara, CA, USA, 2021.
[15] J. Specht and S. Samii, "Urgency-based scheduler for time-sensitive switched Ethernet networks," in *Proc. 28th Euromicro Conf. Real-Time Syst. (ECRTS)*, Jul. 2016, pp. 75–85.
[16] A. B. Ahmed, T. Hirofuchi, and T. Fukai, "FPGA-based network switch architecture supporting credit based shaper for time sensitive networks," in *Proc. IEEE 29th Int. Conf. Emerg. Technol. Factory Autom. (ETFA)*, Sep. 2024, pp. 1–8.
[17] M. Boyer, "Equivalence between the urgency based shaper and asynchronous traffic shaping in time sensitive networking," *Leibniz Trans. Embedded Syst.*, vol. 9, no. 1, pp. 1:1–1:27, 2022.
[18] J. Specht and S. Samii, "Synthesis of queue and priority assignment for asynchronous traffic shaping in switched Ethernet," in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, Dec. 2017, pp. 178–187.
[19] A. Nasrallah, A. S. Thyagaturu, Z. Alharbi, C. Wang, X. Shao, M. Reisslein, and H. Elbakoury, "Performance comparison of IEEE 802.1 TSN time aware shaper (TAS) and asynchronous traffic shaper (ATS)," *IEEE Access*, vol. 7, pp. 44165–44181, 2019.
[20] R. Debnath, P. Hortig, L. Zhao, and S. Steinhorst, "Advanced modeling and analysis of individual and combined TSN shapers in OMNeT++," in *Proc. IEEE 29th Int. Conf. Embedded Real-Time Comput. Syst. Appl. (RTCSA)*, Aug. 2023, pp. 176–185.
[21] Z. Zhou, Y. Yan, M. Berger, and S. Ruepp, "Analysis and modeling of asynchronous traffic shaping in time sensitive networks," in *Proc. 14th IEEE Int. Workshop Factory Commun. Syst. (WFCS)*, Jun. 2018, pp. 1–4.
[22] Z. Zhou, M. S. Berger, S. R. Ruepp, and Y. Yan, "Insight into the IEEE 802.1 qcr asynchronous traffic shaping in time sensitive network," *Adv. Sci., Technol. Eng. Syst. J.*, vol. 4, no. 1, pp. 292–301, 2019.
[23] H. Guo, W. Li, J. Lin, J. Zhou, Q. Xun, S. Zhan, Y. Huang, Y. Wang, and Q. Cheng, "A scalable asynchronous traffic shaping mechanism for TSN with time slot and polling," in *Proc. NOMS IEEE/IFIP Netw. Oper. Manage. Symp.*, May 2023, pp. 1–5.
[24] L. Zhao, P. Pop, and S. Steinhorst, "Quantitative performance comparison of various traffic shapers in time-sensitive networking," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 3, pp. 2899–2928, Sep. 2022.
[25] A. Varga and R. Hornig, "An overview of the OMNeT++ simulation environment," in *Proc. 1st Int. ICST Conf. Simulation Tools Techn. Commun. Netw. Syst.*, 2008, pp. 1–10.
[26] Z. Zhou, M. S. Berger, and Y. Yan, "Mapping TSN traffic scheduling and shaping to FPGA-based architecture," *IEEE Access*, vol. 8, pp. 221503–221512, 2020.
[27] M. Ulbricht, S. Senk, H. K. Nazari, H.-H. Liu, M. Reisslein, G. T. Nguyen, and F. H. P. Fitzek, "TSN-FlexTest: Flexible TSN measurement testbed (extended version)," 2022, *arXiv:2211.10413*.
[28] C. Pfefferlex, F. Wiedner, and C. Schwarzenberg, "IEEE 802.1QCR asynchronous traffic shaping with Linux traffic control," in *Proc. Seminar Innov. Internet Technol. Mobile Commun. (IITM)*, Jan. 2022, pp. 11–14.
[29] M. Bosk, F. Rezabek, K. Holzinger, A. G. Marino, A. A. Kane, F. Fons, J. Ott, and G. Carle, "Methodology and infrastructure for TSN-based reproducible network experiments," *IEEE Access*, vol. 10, pp. 109203–109239, 2022.
[30] O. Yasin, Y. Kobayashi, T. Yamaura, and T. Maegawa, "Software-based time-aware shaper for time-sensitive networks," *IEICE Trans. Commun.*, vol. E103.B, no. 3, pp. 167–180, 2020.
[31] C. Xue, T. Zhang, and S. Han, "Towards cost-effective real-time high-throughput end station design for time-sensitive networking (TSN)," in *Proc. 61st ACM/IEEE Design Autom. Conf.*, Jun. 2024, pp. 1–6.
[32] C. Xue, T. Zhang, A. Loveless, and S. Han, "Supporting deterministic traffic on standard NICs," 2025, *arXiv:2506.17877*.
[33] M. Ulbricht, S. Senk, H. K. Nazari, H.-H. Liu, M. Reisslein, G. T. Nguyen, and F. H. P. Fitzek, "TSN-FlexTest: Flexible TSN measurement testbed," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 2, pp. 1387–1402, Apr. 2024.
[34] A. B. Ahmed, T. Hirofuchi, and T. Fukai, "EFCC: Ethernet frame crafter & capture for TSN research," in *Proc. IEEE 50th Conf. Local Comput. Netw. (LCN)*, Oct. 2025, pp. 1–9.

TAKAHIRO HIROFUCHI received the Ph.D. degree in engineering from the Graduate School of Information Science, Nara Institute of Science and Technology (NAIST), in March 2007. He is currently a Senior Research Scientist and a Research Group Leader with the National Institute of Advanced Industrial Science and Technology (AIST), Japan. He is working on the co-design of software and hardware for edge and cloud computing. He is also an Expert of operating systems, virtual machine, and network technologies.

AKRAM BEN AHMED (Member, IEEE) received the Ph.D. degree in computer science and engineering from The University of Aizu, Japan, in 2015. He is currently a Senior Research Scientist with the National Institute of Advanced Industrial Science and Technology, Tokyo, Japan. His research interests include on-chip interconnection networks, reliable and fault-tolerant systems, and ultra-low-power embedded systems.

TAKAAKI FUKAI received the B.S. degree in engineering from Okayama University, Japan, in 2013, and the M.S. and Ph.D. degrees in engineering from the University of Tsukuba, Japan, in 2015 and 2018, respectively. He is currently a Senior Research Scientist with the National Institute of Advanced Industrial Science and Technology (AIST), Japan. His research interests include operating systems, virtualization, and cloud computing.